

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

```
cryptoquant_dump.py
=====
```

.رو می‌کشد بیرون که بعداً بدی برای تحقیق CryptoQuant Data API کل دیتاست

کاتالوگ (CryptoQuant (/v1/discovery/endpoints) کشفِ خودِ endpoint خودش از
 «ها رو پول می‌کنه - پس واقعاً همش endpoint کامل پلن تو رو می‌گیره و بعد همه‌ی اون

: اجرا (سریع)

```
export CQ_API_TOKEN="هیچ وقت توکن رو داخل کد ننویس"
pip install requests
python cryptoquant_dump.py --mode all --from 20180101 --window day --zip
```

Modes:

```
discover : -> catalog.json ها رو بگیر endpoint فقط کاتالوگ
pull      : discover اگه کاتالوگ نباشه خودش) های کاتالوگ رو بکش endpoint دیتای
all       : discover + pull (پیشفرض)
```

خروجی (./cq_data/ پیشفرض):

```
catalog.json      ها (مطابق پلن تو) endpoint کاتالوگ کامل
catalog_index.txt لیست خوانا
data/<asset>/<category>/<metric>.csv یک CSV برای هر endpoint
manifest.json/.csv چی کشیده شد: تعداد ردیف، بازه تاریخ، وضعیت، خطاها
latest_snapshot.csv آخرین مقدار هر متریک (جمع و جور، راحت برای پیست کردن)
cq_data.zip (با --zip) همه چی یکجا که آپلود کنی اینجا برای تحقیق
```

نکته‌ها:

```
- Authorization: Bearer <token> احراز هویت = هدر
- فرانتیرت CPU-mined کوین‌های BTC/ETH/XRP/TRON/stablecoins/ERC-20/alts: پوشش
  لایه ماکرو/فلوئه + BTC نیستن - این برای هسته CryptoQuant روی (XEL, DIL, TAO ...)
- --overwrite: های پر و موجود رد می‌شن مگه با CSV: قابل ادامه
"""
```

```
import argparse
import csv
import datetime as dt
import json
import os
import sys
import time
import zipfile
from pathlib import Path
```

```

try:
    import requests
except ImportError:
    sys.exit("نیاز به requests: pip install requests")

BASE = "https://api.cryptoquant.com/v1"
DISCOVERY_PATH = "/discovery/endpoints"
TOKEN_ENV = "CQ_API_TOKEN"

# ----- #
# HTTP
# ----- #
def make_session(token: str) -> requests.Session:
    s = requests.Session()
    s.headers.update(
        {
            "Authorization": f"Bearer {token}",
            "Accept": "application/json",
            "User-Agent": "cq-dump/1.0",
        }
    )
    return s

def cq_get(session, path, params, max_retries=5, base_sleep=1.0):
    """GET با backoff. خروجی: (json|None, error_str|None)."""
    url = BASE + path if path.startswith("/") else f"{BASE}/{path}"
    for attempt in range(max_retries + 1):
        try:
            r = session.get(url, params=params, timeout=90)
        except requests.RequestException as e:
            if attempt == max_retries:
                return None, f"network: {e}"
            time.sleep(base_sleep * (2 ** attempt))
            continue

        if r.status_code == 200:
            try:
                return r.json(), None
            except ValueError:
                return None, "bad-json"

        if r.status_code == 429: # rate limited
            wait = base_sleep * (2 ** attempt) * 3
            time.sleep(wait)
            continue

```

```

        if r.status_code in (400, 401, 403, 404, 422):
            # فایده نداره retry خطای دائمی برای این درخواست؛
            return None, f"http {r.status_code}: {r.text[:160]}"
        # 5xx و بقیه -> retry
        if attempt == max_retries:
            return None, f"http {r.status_code}"
        time.sleep(base_sleep * (2 ** attempt))
    return None, "max-retries"

# ----- #
# Catalog + params
# ----- #
def fetch_catalog(session):
    data, err = cq_get(session, DISCOVERY_PATH, {"format": "json"})
    if err:
        raise RuntimeError(f"کاتالوگ نیومد: {err}")
    items = (data or {}).get("result", {}).get("data", [])
    out = []
    for it in items:
        path = (it.get("path") or "").strip()
        if not path:
            continue
        out.append({"path": path, "parameters": it.get("parameters") or ""})
    return out

def split_path(path):
    """'/btc/exchange-flows/reserve' -> ('btc','exchange-flows','reserve')."""
    segs = [s for s in path.strip("/").split("/") if s]
    asset = segs[0] if segs else "misc"
    category = segs[1] if len(segs) > 1 else "general"
    metric = "_".join(segs[2:]) if len(segs) > 2 else (segs[-1] if segs else "dat")
    return asset, category, metric

def build_params(param_spec, args, include_time=True):
    """اومده + پیشفرضهای امن endpoint اون spec پارامتر می‌سازه؛ فقط چیزایی که تو"""
    spec = (param_spec or "").lower()
    p = {"format": "json", "limit": str(args.limit)}
    spec_empty = not spec.strip()

    if include_time and ("window" in spec or spec_empty):
        p["window"] = args.window
    if include_time and ("from" in spec or spec_empty):
        p["from"] = args.date_from
    if include_time and ("to" in spec or spec_empty):

```

```

        p["to"] = args.date_to
    if "exchange" in spec:
        p["exchange"] = "all_exchange"
    if "miner" in spec:
        p["miner"] = "all_miner"
    if "symbol" in spec:
        p["symbol"] = "all_symbol"
    if "token" in spec:
        p["token"] = args.token
    return p

# ----- #
# Save helpers
# ----- #
def write_csv(rows, fpath: Path):
    if not rows:
        return 0
    keys = []
    seen = set()
    for row in rows:
        for k in row.keys():
            if k not in seen:
                seen.add(k)
                keys.append(k)
    fpath.parent.mkdir(parents=True, exist_ok=True)
    with fpath.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        for row in rows:
            w.writerow(row)
    return len(rows)

def date_field(row):
    for k in ("date", "datetime", "time", "blockheight", "timestamp"):
        if k in row:
            return str(row[k])
    return ""

# ----- #
# Main pull
# ----- #
def pull_all(session, catalog, args):
    out = Path(args.out)
    data_dir = out / "data"

```

```

manifest = []
snapshot = []
n_ok = n_skip = n_err = 0

asset_filter = {a.strip().lower() for a in args.assets.split(",")} if args.as
cat_filter = {c.strip().lower() for c in args.categories.split(",")} if args.

total = len(catalog)
for i, ep in enumerate(catalog, 1):
    path, spec = ep["path"], ep["parameters"]
    asset, category, metric = split_path(path)

    if asset_filter and asset not in asset_filter:
        continue
    if cat_filter and category not in cat_filter:
        continue

    fpath = data_dir / asset / category / f"{metric}.csv"
    if fpath.exists() and fpath.stat().st_size > 0 and not args.overwrite:
        n_skip += 1
        print(f"[{i}/{total}] skip (exists) {path}")
        continue

    # ر ه window/from/to خورد بدون xxتلاش اول با پارامترهای زمان دار، اگه خطای ۴
    params = build_params(spec, args, include_time=True)
    data, err = cq_get(session, path, params, args.retries, args.sleep)
    if err and err.startswith("http 4"):
        params = build_params(spec, args, include_time=False)
        data, err = cq_get(session, path, params, args.retries, args.sleep)

    if err:
        n_err += 1
        manifest.append(
            {"path": path, "asset": asset, "category": category, "metric": me
             "rows": 0, "first": "", "last": "", "status": "error", "detail":
            )
        print(f"[{i}/{total}] ERR {path} -> {err}")
        time.sleep(args.sleep)
        continue

    rows = (data or {}).get("result", {}).get("data", []) or []
    n = write_csv(rows, fpath)
    first = date_field(rows[-1]) if rows else "" # آخرین = قدیمی‌ترین API
    last = date_field(rows[0]) if rows else ""
    manifest.append(
        {"path": path, "asset": asset, "category": category, "metric": metric
         "rows": n, "first": first, "last": last,

```

```

        "status": "ok" if n else "empty", "detail": ""}
    )
    if rows:
        snap = {"path": path, "asset": asset, "category": category, "metric":
            "last_date": last, "last_row": json.dumps(rows[0], ensure_ascii=False)}
        snapshot.append(snap)
        n_ok += 1
    print(f"[{i}/{total}] ok    {path}    rows={n}")
    time.sleep(args.sleep)

# manifest + snapshot
out.mkdir(parents=True, exist_ok=True)
(out / "manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2))
write_csv(manifest, out / "manifest.csv")
if snapshot:
    write_csv(snapshot, out / "latest_snapshot.csv")

print("\n- خلاصه -")
print(f"    ok={n_ok}    skip={n_skip}    err={n_err}    total_endpoints={total}")
print(f"    خروجی: {out.resolve()}")
return manifest

def zip_output(args):
    out = Path(args.out)
    zpath = out.with_suffix(".zip")
    with zipfile.ZipFile(zpath, "w", zipfile.ZIP_DEFLATED) as z:
        for p in out.rglob("*"):
            if p.is_file():
                z.write(p, p.relative_to(out.parent))
    print(f"    zip: {zpath.resolve()}")

# ----- #
# CLI
# ----- #

def parse_args(argv=None):
    today = dt.date.today().strftime("%Y%m%d")
    ap = argparse.ArgumentParser(description="CryptoQuant بیرون کل دیتاست")
    ap.add_argument("--mode", choices=["discover", "pull", "all"], default="all")
    ap.add_argument("--out", default="cq_data", help="پوشه خروجی")
    ap.add_argument("--from", dest="date_from", default="20180101", help="YYYYMMDD")
    ap.add_argument("--to", dest="date_to", default=today, help="YYYYMMDD")
    ap.add_argument("--window", default="day", choices=["min", "hour", "day", "bl"])
    ap.add_argument("--limit", type=int, default=100000, help="تعداد ردیف هر درخواست")
    ap.add_argument("--assets", default="", help="(خالی=همه) btc,eth فیلتر مثل")
    ap.add_argument("--categories", default="", help="(خالی=همه) فیلتر دسته")

```

```

ap.add_argument("--token", default="usdt", help="ای استیبل پایش فرض param=token")
ap.add_argument("--sleep", type=float, default=0.6, help="ن درخواستها (ثانیه)")
ap.add_argument("--retries", type=int, default=5)
ap.add_argument("--overwrite", action="store_true", help="رو هم دوباره بکش CSV")
ap.add_argument("--zip", action="store_true", help="کن zip آخرش همه چی رو")
return ap.parse_args(argv)

def main(argv=None):
    args = parse_args(argv)
    token = os.environ.get(TOKEN_ENV, "").strip()
    if not token:
        sys.exit(
            f"توکن پیدا نشد. اول این رو بزن\n"
            f' export {TOKEN_ENV}="توکننت"\n'
            f"cryptoquant.com/settings/api) پروفایلت تو API از تب)"
        )

    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    session = make_session(token)

    # کاتالوگ
    cat_path = out / "catalog.json"
    if args.mode in ("discover", "all") or not cat_path.exists():
        print("... ها گرفتن کاتالوگ")
        catalog = fetch_catalog(session)
        cat_path.write_text(json.dumps(catalog, ensure_ascii=False, indent=2), en)
        (out / "catalog_index.txt").write_text(
            "\n".join(f"{{e['path']}}    [{{e['parameters']}}]" for e in catalog), en
        )
        print(f" {len(catalog)} endpoint شد پیدا -> {cat_path}")
    else:
        catalog = json.loads(cat_path.read_text(encoding="utf-8"))

    if args.mode == "discover":
        return

    pull_all(session, catalog, args)
    if args.zip:
        zip_output(args)

if __name__ == "__main__":
    main()

```

